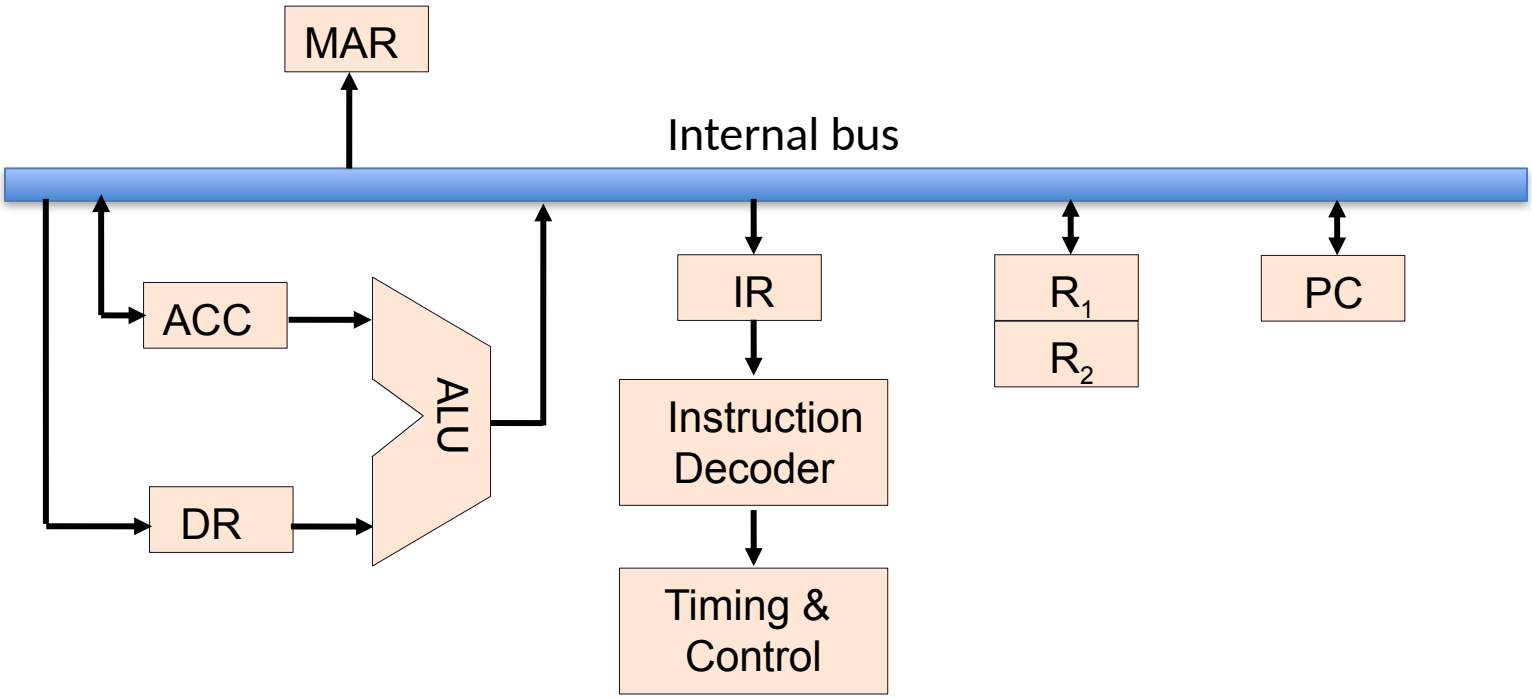
Control Unit

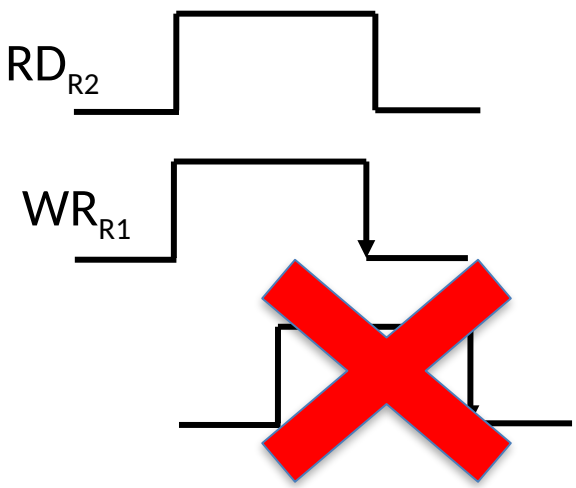
By
Dr. Sk Subidh Ali

Computer Datapath



Timing Signal

MOV R_1, R_2



MVI

ADI

JMP

Phases of instruction execution

- Instruction fetch
- Instruction decode
- Instruction execute

Possible instructions

ADD R_1 ACC ↯ ACC + R_1

CMP ACC ↯ $\overline{ACC}$

SUB, MUL, DIV

AND R_1 ACC ↯ ACC $\wedge$ R_1

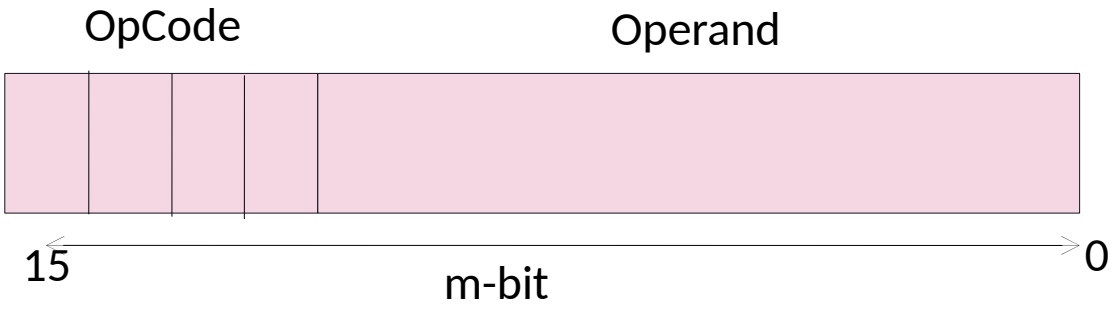
NAND and all logical operation

MOV R_1, R_2 MOV R_2, R_1

MOV ACC, R MOV R, ACC

MOV ACC, M MOV M, ACC

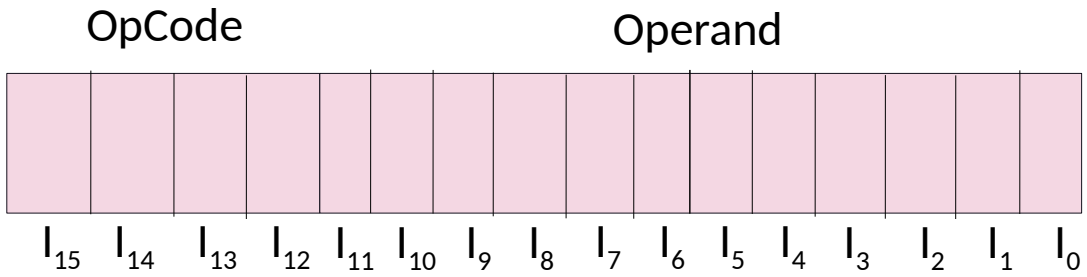
Instruction Format



There are two instruction groups

- Register reference
- Memory reference

Instruction Format



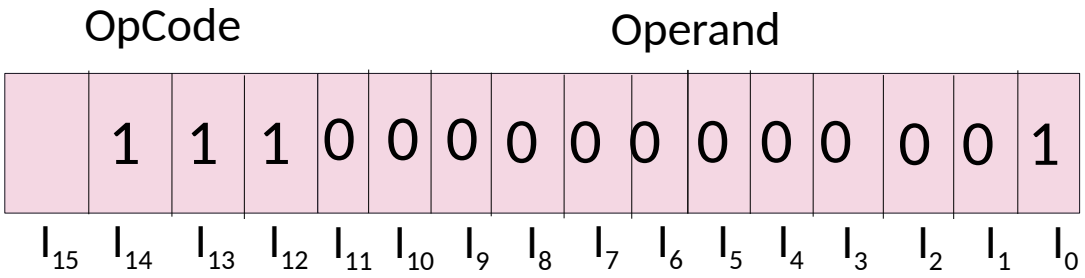
There are two instruction group

- **Register reference**
- Memory reference

No operand address required

12 bits of operand can be used
to refer different register instructions

Instruction Format



I_0 ADD R_1 ACC ACC + R_1

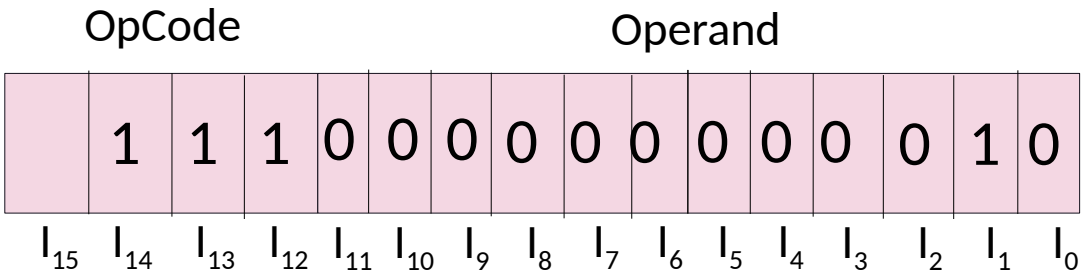
There are two instruction group

- **Register reference**
- Memory reference

No operand address required

12 bits of operand can be used
to refer different register instructions

Instruction Format



I_0 ADD R_1 ACC ↯ ACC + R_1

I_1 CMP ACC ↯ ACC

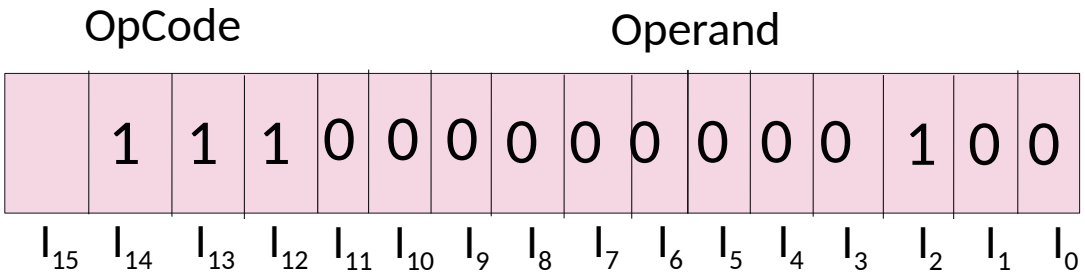
There are two instruction group

- **Register reference**
- Memory reference

No operand address required

12 bits of operand can be used
to refer different register instructions

Instruction Format



I_0 ADD R_1 ACC ↯ ACC + R_1

I_1 CMP ACC ↯ ACC

I_2 AND R_1 ACC ↯ ACC ∧ R_1

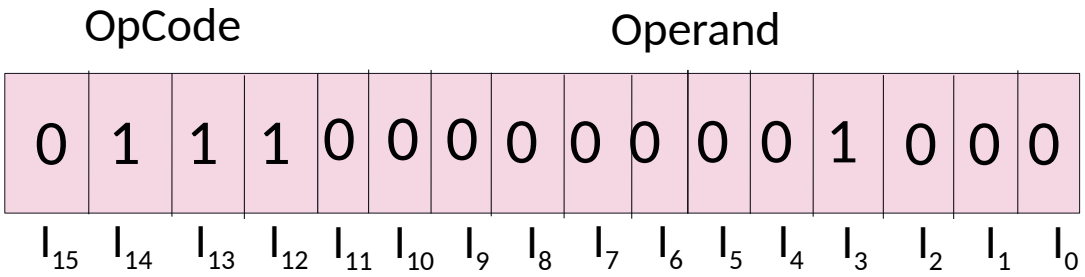
There are two instruction group

- **Register reference**
- Memory reference

No operand address required

12 bits of operand can be used
to refer different register instructions

Instruction Format



There are two instruction group

- **Register reference**
- Memory reference

No operand address required

12 bits of operand can be used to refer different register instructions

I_0 ADD R_1 ACC$\leftarrow$ ACC + R_1

I_1 CMP ACC$\leftarrow$ $\overline{ACC}$

I_2 AND R_1 ACC$\leftarrow$ ACC $\wedge$ R_1

I_3 MOV R_1,R_2 $R_1\leftarrow R_2$

I_4 MOV R_2,R_1 $R_2\leftarrow R_1$

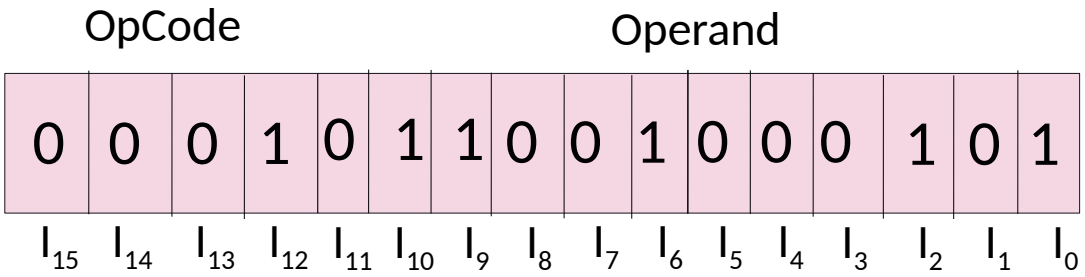
I_5 MOV ACC,R_1 ACC$\leftarrow R_1$

I_6 MOV R_1, ACC $R_1\leftarrow$ ACC

I_7 MOV ACC,R_2 ACC$\leftarrow R_2$

I_8 MOV R_2, ACC $R_2\leftarrow$ ACC

Instruction Format



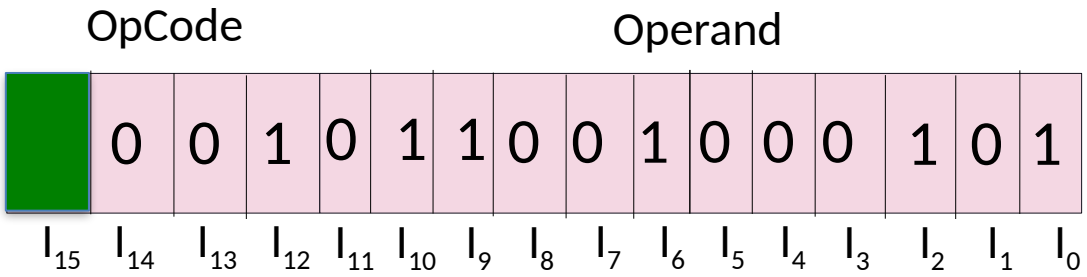
There are two instruction group

- **Register reference**
- Memory reference

0001	MOV ACC, [M]	001 0x234
0010	MOV [M], ACC	010 0x434
0011	JMP	011 0x123

12 bits of operand can be memory address

Instruction Format



There are two instruction group

- **Register reference**
- **Memory reference**

I_{15} can be used for immediate addressing

1001	MVI ACC, 0x02
1010	ADI ACC, 0x23
1011	ANI ACC, 0x03

1001 0x02
1010 0x23
1011 0x03

8 bits of operand can be taken for immediate value

Instruction Phases

Instr^{n} fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: Depends on the instruction

Instruction Phases

ADD R_1 : ACC<=ACC+R_1

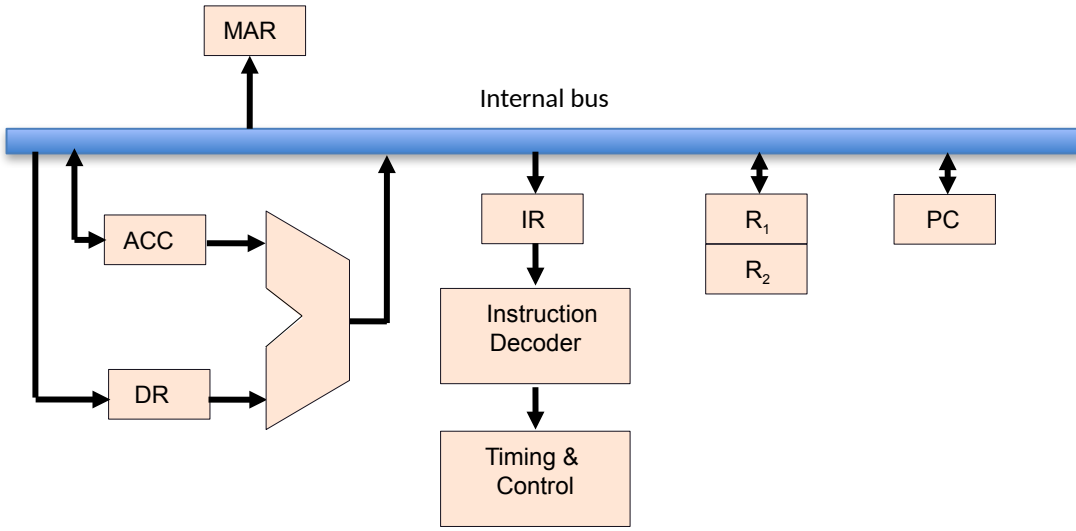
T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR)

T_3: DR<= R1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0



Instruction Phases

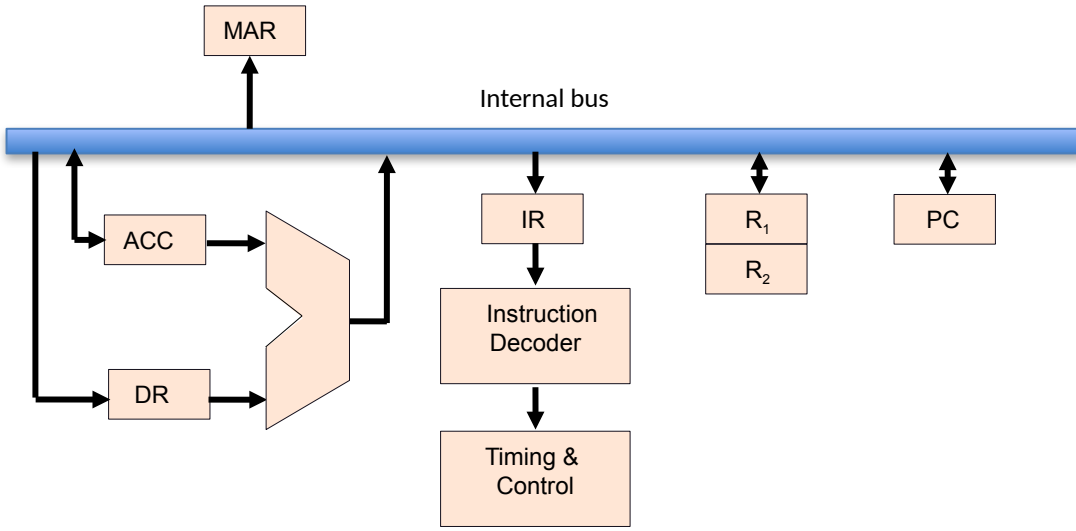
MOV R_1,R_2 : **R_1<=R_2**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR)

T_3: R_2<= R_1 , T_0



Instruction Phases

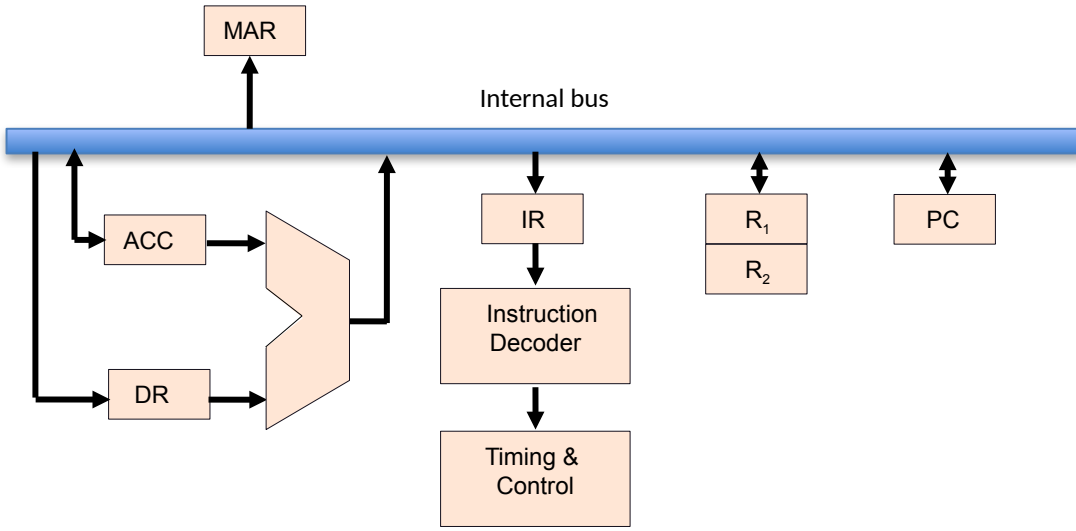
MOV ACC, [M] : **ACC<=[M]**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

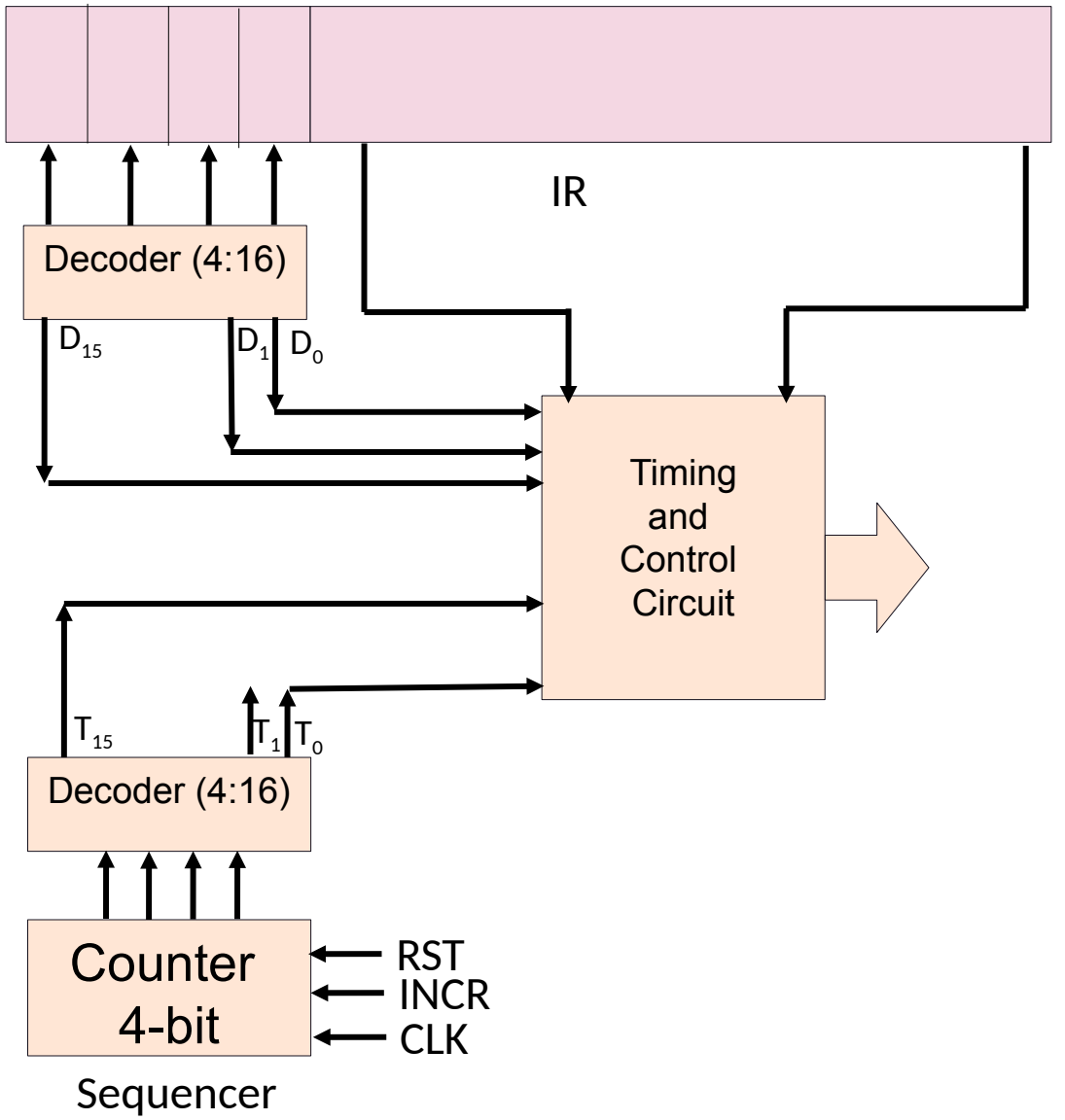
T_2: DECD(IR), MAR<=IR_{11to0}

T_3: ACC<= M[MAR], T_0



We need timing sequence and control signal

Control Hardware



Control Signals

ADD R_1 : **ACC<=ACC+R_1**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

PC: RD = T_0+

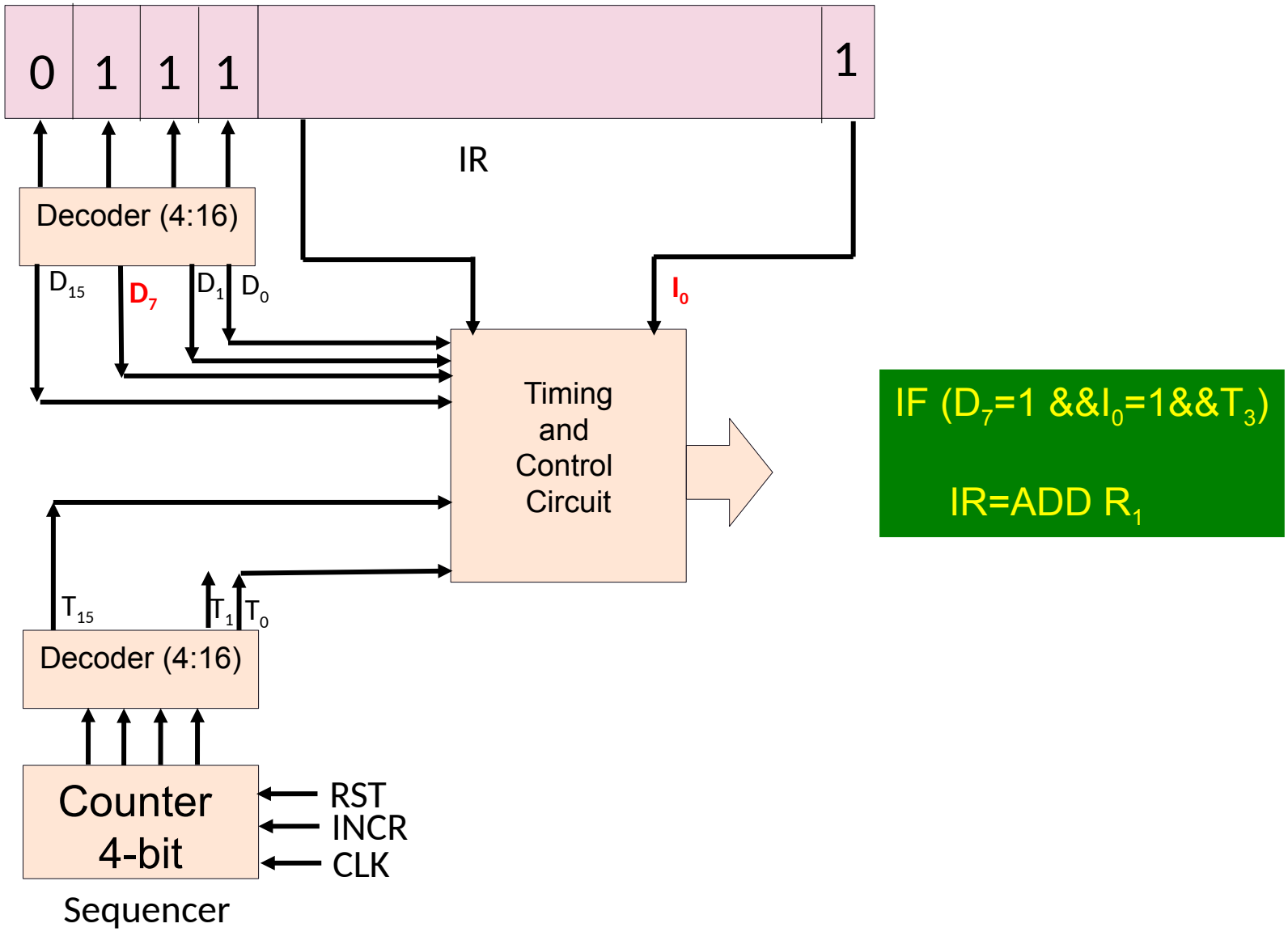
MAR: WR = T_0+T_2+

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

Control Hardware



Control Signals

ADD R_1 : **ACC<=ACC+R_1**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : **R_1<=R_2**

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+

ALU: ADD = $T_4D_7I_0$+

SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +

R_2: RD = $T_3D_7I_3$+

Control Signals

ADD R_1 : ACC<=ACC+R_1

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : R_1<=R_2

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+

ALU: ADD = $T_4D_7I_0$+

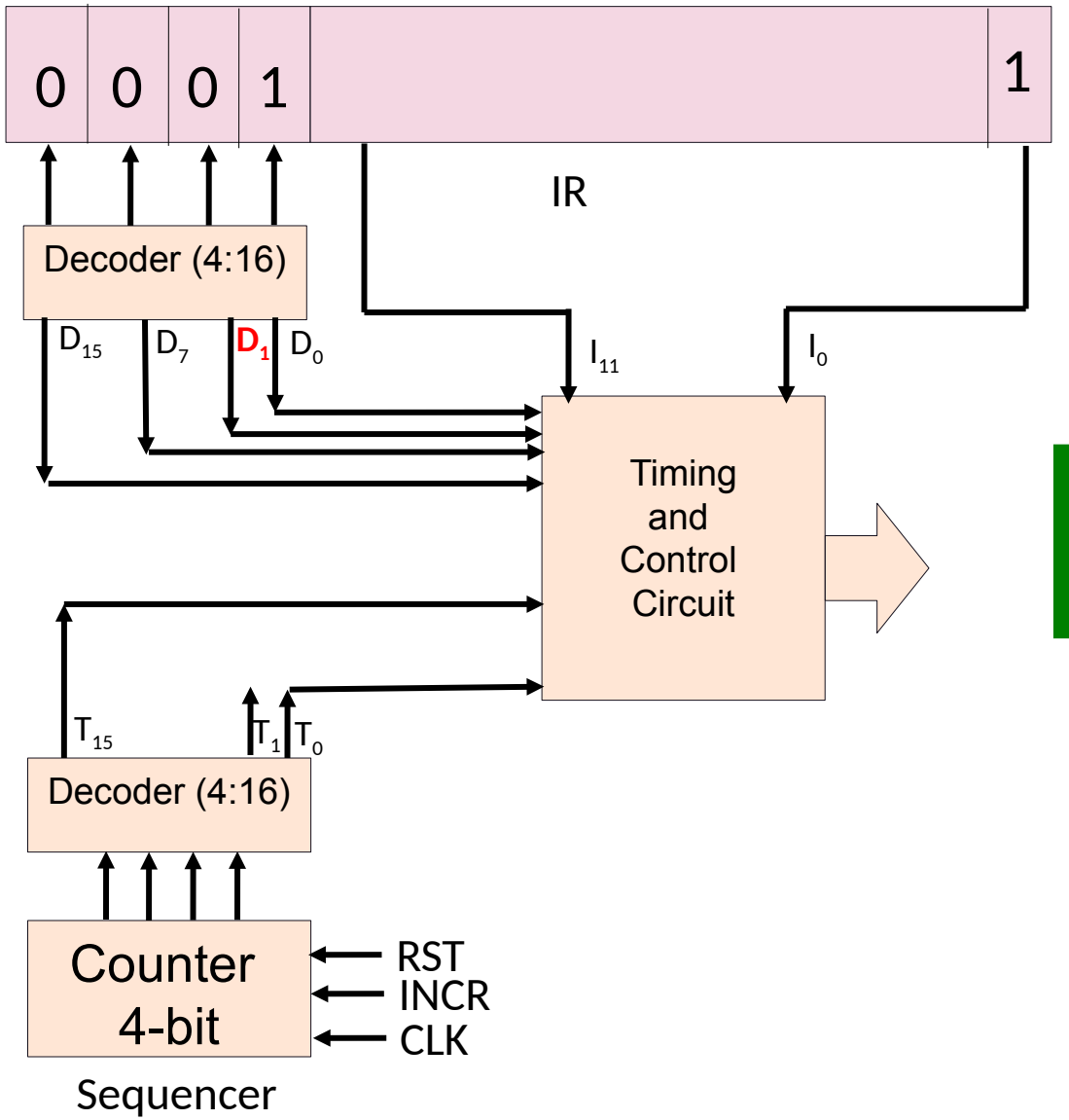
SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +$T_3D_7I_3$

R_2: RD = $T_3D_7I_3$+

MOV ACC, [M] : ACC<=[M]

Control Hardware



0001 MOV ACC, [M]

IF (D_1=1 &&T_3)

IR=MOV ACC, [M]

Control Signals

ADD R_1 : ACC<=ACC+R_1

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : R_1<=R_2

MOV ACC, [M] : ACC<=[M]

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+

ALU: ADD = $T_4D_7I_0$+

SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +$T_3D_7I_3$

R_2: RD = $T_3D_7I_3$+

Control Signals

ADD R_1 : **ACC<=ACC+R_1**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : **R_1<=R_2** **MOV ACC, [M] : ACC<=[M]**

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+ T_3D_1 +

ALU: ADD = $T_4D_7I_0$+

SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +$T_3D_7I_3$

R_2: RD = $T_3D_7I_3$+

M: MR = T_3D_1+

Control Signals

ADD R_1 : **ACC<=ACC+R_1**

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : **R_1<=R_2** **MOV ACC, [M] : ACC<=[M]**

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+ T_3D_1 +

ALU: ADD = $T_4D_7I_0$+

SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +$T_3D_7I_3$+

T_3D_1+

R_2: RD = $T_3D_7I_3$+

M: MR = T_3D_1+

Control Signals

ADD R_1 : ACC<=ACC+R_1

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MOV R_1,R_2 : R_1<=R_2

MOV ACC, [M] : ACC<=[M]

PC: RD = T_0+

MAR: WR = T_0+ T_2

IR: WR = T_1+

PC: INR = T_1+

IR: RD = T_2+

R_1: RD = $T_3D_7I_0$+

WR = $T_3D_7I_3$+

DR: WR = $T_3D_7I_0$+

ACC: WR = $T_4D_7I_0$+ T_3D_1 +

ALU: ADD = $T_4D_7I_0$+

SEQCNTR: INR = T_0+T_1+T_2 +$T_3D_7I_0$

CLR = $T_4D_7I_0$ +$T_3D_7I_3$+

T_3D_1+

R_2: RD = $T_3D_7I_3$+

M: MR = T_3D_1+T_1

Mem. read during instrⁿ fetch

Mirprogrammed Control Unit

CS2600 (3003)

By

Dr. Sk Subidh Ali

15-Feb-2017

Micro-programmed Control Unit

Instruction fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

MAR: WR => C_0
PC: RD => C_1
IR: WR => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5

Micro-programmed Control Unit

Instruction fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5

	C_5	C_4	C_3	C_2	C_1	C_0
T_0:	0	0	0	0	1	1
T_1:	0	1	1	1	0	0
T_2:	1	0	0	0	0	1

How to sequence??
T_0=>T_1=>T_2

Micro-programmed Control Unit

Instruction fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5

	C_5	C_4	C_3	C_2	C_1	C_0	BA				M
T_0:	0	0	0	0	1	1	0	0	0	1	1
T_1:	0	1	1	1	0	0	0	0	1	0	1
T_2:	1	0	0	0	0	1	x	x	x	x	0

BA: Branch address
M: mode of address

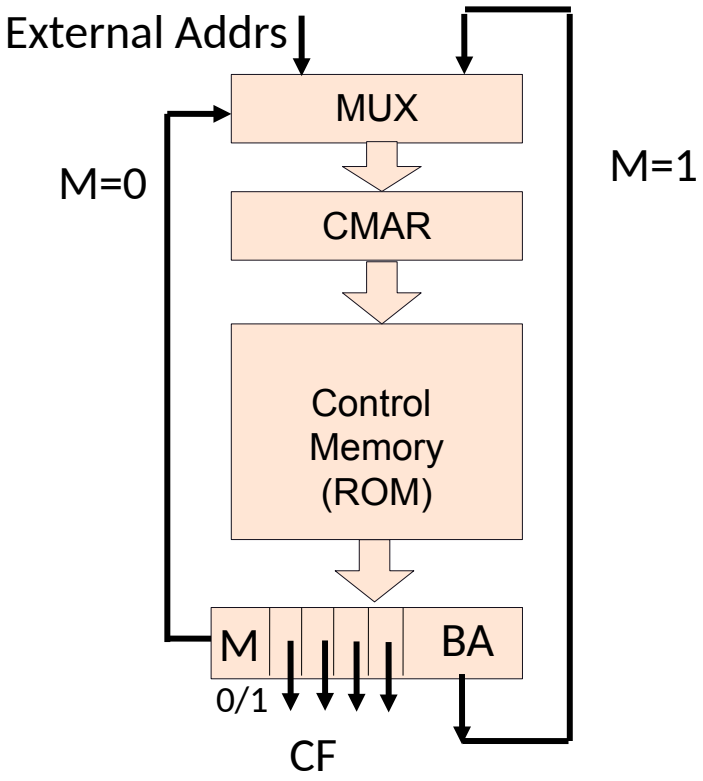
Micro-programmed Control Unit

Instruction fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}



MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5

	C_5	C_4	C_3	C_2	C_1	C_0	BA				M
T_0:	0	0	0	0	1	1	0	0	0	1	1
T_1:	0	1	1	1	0	0	0	0	1	0	1
T_2:	1	0	0	0	0	1	x	x	x	x	0

What about complete instruction????

CMAR: Control memory address register
CF: control field/control signals
BA: Branch address

Micro-programmed Control Unit

Instruction fetch

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5

Micro-programmed Control Unit

ADD R_1 ACC<=ACC+R_1

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5
R_1: RD => C_6
DR: LD => C_7
ACC: LD => C_8
ALU: ADD=> C_9
ALU: RD => C_{10}

	C_{10}	C_9	C_8	C_7	C_6	C_5	C_4	C_3	C_2	C_1	C_0	BA	M
T_0:	0	0	0	0	0	0	0	0	0	1	1	0 0	0 1 1
T_1:	0	0	0	0	0	0	1	1	1	0	0	0 0	1 0 1
T_2:	0	0	0	0	0	1	0	0	0	0	1	L	0

Micro-programmed Control Unit

ADD R_1 ACC<=ACC+R_1

T_0: MAR<= PC

T_1: IR<= [MAR], PC<=PC+1

T_2: DECD(IR), MAR<=IR_{11to0}

T_3: DR<= R_1

T_4: ACC<= ALU_{ADD}(ACC, DR), T_0

MAR: LD => C_0
PC: RD => C_1
IR: LD => C_2
M: RD => C_3
PC: INR => C_4
IR: RD => C_5
R_1: RD => C_6
DR: LD => C_7
ACC: LD => C_8
ALU: ADD=> C_9
ALU: RD => C_{10}

	C_{10}	C_9	C_8	C_7	C_6	C_5	C_4	C_3	C_2	C_1	C_0	BA	M
T_0:	0	0	0	0	0	0	0	0	0	1	1	0 0 0 1	1
T_1:	0	0	0	0	0	0	1	1	1	0	0	0 0 1 0	1
T_2:	0	0	0	0	0	1	0	0	0	0	1	L	0
T_3:	0	0	0	1	1	0	0	0	0	0	0	L+1	1
T_4:	1	1	1	0	0	1	0	0	0	0	1	L+2	0